Arbitrage spatial HFT sous cTrader Open API

en Rust

Spécifications d'ingénierie logicielle pour exploiter une fenêtre d'opportunité de **150 ms** sur l'arbitrage spatial multi-LP. Face à un Last Look asymétrique de **40 ms**, l'application cliente intègre un **radar Last Look local** — jumeau numérique du LP dérivé du modèle de Cartea, Jaimungal & Walton (2018) — qui prédit la probabilité de rejet avant l'envoi de l'ordre. Vérifié contre les fichiers .proto officiels **spotware/openapi-proto-messages**.

150 ms
FENÊTRE OPPORTUNITÉ

40 ms
LAST LOOK LP

65 ms
EXPOSITION MARCHÉ

10 µs
JITTER OS CIBLE

ENVIRONNEMENT
Linux low-jitter — Equinix LD4

STACK
Rust · Tokio · prost · cTrader Open API

DATE
10 novembre 2026

RÉFÉRENCE
ARB-HFT-CTR-V4-2026

Table des matières

1. Synthèse exécutive 2

2. Analyse chronométrique et budget de latence 2

3. Architecture logique Rust : découplage synchrone / asynchrone 3

4. Routage ECN, IOC et MARKET_RANGE 4

5. Code source Rust : construction d'un ordre protégé 5

6. Matrice opérationnelle de gestion des événements asynchrones 8

7. Gestion d'erreurs, résilience et kill switch 9

8. Note de conformité et avertissement de risque 10

9. Modélisation académique du Last Look 10

10. Le radar Last Look local : un jumeau numérique du LP 12

11. Équilibre multi-places et stratégie de routage 18

1. Synthèse exécutive

Ce document définit les spécifications d'ingénierie logicielle requises pour exploiter une stratégie d'**arbitrage spatial multi-LP** sur cTrader Open API, dans une fenêtre d'opportunité de **150 ms**. L'anomalie de prix arbitrée provient d'un décalage transitoire entre les cotations de plusieurs fournisseurs de liquidité (LP) sur un même symbole — typiquement un LP principal encore à l'ancien prix pendant que le marché a déjà bougé. La stratégie doit donc frapper deux jambes (achat chez le LP en retard, vente chez le LP à jour) et sortir avant la convergence des prix.

Face à un mécanisme de **Last Look asymétrique fixe de 40 ms** appliqué par les LP — fenêtre pendant laquelle le client est engagé mais le LP conserve le droit de refuser l'exécution — l'application cliente doit éliminer toute attente synchrone via un modèle découplé asynchrone développé en **Rust** avec l'exécuteur Tokio. Le protocole binaire cTrader impose en parallèle des contraintes d'exécution strictes : l'instruction **IOC (Immediate-Or-Cancel)** combinée au typage d'ordre **MARKET_RANGE** forme la barrière de protection principale contre une exécution à perte si le prix dérive durant le Last Look.

L'architecture cible s'appuie sur un serveur Linux optimisé en jitter OS (< 10 µs), idéalement colocalisé en périphérie du data center cTrader (Equinix LD4 Slough ou équivalent métropolitain). Le budget de latence cumulé aller-simple est de **~42 ms**, soit environ **28 %** de la fenêtre globale. Après exécution de l'ordre d'entrée, il ne reste qu'environ **107 ms** de validité structurelle, dont **~65 ms** réellement exposées au marché une fois déduit le transit de l'ordre de sortie et le second Last Look potentiel.

150 ms	~42 ms	28 %	65 ms
Fenêtre d'opportunité	Latence aller-simple	Budget consommé	Exposition marché

2. Analyse chronométrique et budget de latence

Pour une fenêtre d'opportunité globale de 150 ms, la latence de transit aller-simple (one-way) et le temps de traitement de l'ordre par les contreparties consomment une part incompressible de la marge temporelle. L'architecture est modélisée avec un serveur Linux optimisé en périphérie du data center principal de cTrader (ex. Equinix LD4 Slough), ce qui induit des coûts de routage réseau métropolitain non négligeables mais nettement inférieurs à un hébergement cloud public.

Couche / Segment	Lat. min	Lat. max	Type d'impact applicatif
Couche 1 : Sérialisation Rust Protobuf + Socket Kernel	`0,02 ms`	`0,05 ms`	Déterministe (jitter OS < 10 µs)
Couche 2 : Réseau fibre externe + Routage Edge Cloud	`0,40 ms`	`1,50 ms`	Variable (gigue de transport réseau)

Couche / Segment	Lat. min	Lat. max	Type d'impact applicatif
Couche 3 : Moteur de risque du courtier (in-memory)	`0,10 ms`	`0,30 ms`	Dépend de la charge globale du broker
Couche 4 : Conversion FIX + Routage LP (Last Look)	`40,10 ms`	`40,50 ms`	Fixe (délai de décision asymétrique du LP)
Total cumulé (arrivée / remplissage)	**`40,62 ms`**	**`42,35 ms`**	**Budget consommé : ~28 % de la fenêtre**

Tableau 1 — Décomposition du budget de latence aller-simple (T0 → FILLED)

2.1 Analyse du budget résiduel

Une fois l'ordre d'entrée exécuté (statut FILLED) à T + 42 ms, le système ne dispose plus que de ~107 ms de validité structurelle avant que l'anomalie de prix ne se referme. À cela doit être soustrait le temps de transit de l'ordre de sortie (fermeture de position), qui subit les mêmes contraintes réseau (~2 ms) et un potentiel second Last Look de 40 ms. Le temps d'exposition réel au marché est donc limité à un maximum théorique de **65 ms**.

Point d'attention. Le second Last Look n'est pas systématique : si l'ordre de sortie est envoyé en MARKET BOC (Book-or-Cancel) chez un LP ECN véritable sans Last Look (rare mais existant sur certains venues MTF), la latence de sortie tombe à ~2,5 ms et l'exposition marché remonte à ~103 ms. La stratégie doit dynamiquement sélectionner le venue de sortie en fonction du profil Last Look connu par LP, maintenu dans une table de configuration chaude.

L'incertitude principale porte sur la **couche 2** (réseau métropolitain). Une gigue de 1 ms sur cette couche consomme directement 0,7 % de la fenêtre. Le serveur doit idéalement être colocalisé à moins de 10 km du point d'accès cTrader, avec une fibre sombre dédiée et un switch cut-through (ex. Arista 7050X3, Mellanox Spectrum). Au-delà de 50 km, la stratégie devient marginale et dépend de la persistance moyenne de l'anomalie.

3. Architecture logique Rust : découplage synchrone / asynchrone

Pour préserver l'intégrité du moteur de détection d'anomalies, le thread principal de calcul ne doit jamais être bloqué par l'attente du flux réseau TCP (I/O bound). L'attente des 40 ms imposée par le Last Look du LP est gérée de manière totalement transparente par un modèle asynchrone structuré autour de **tâches distribuées Tokio** (green threads) interconnectées par des canaux à passage de messages (**MPSC channels**).

3.1 Le mécanisme de corrélation asynchrone (clientMsgId / clientOrderId)

Puisque le socket réseau envoie des rafales de requêtes en continu, le serveur cTrader désynchronise les réponses en fonction du temps de réponse de chaque LP. Pour corréler la réponse binaire reçue au bout de 40 ms avec l'ordre d'arbitrage initial, l'application Rust exploite deux champs distincts du protocole ouvert cTrader : **clientMsgId** (sur l'enveloppe `ProtoMessage`, champ 3) et **clientOrderId** (sur `ProtoOANewOrderReq`, champ 18, MaxLength = 50).

La distinction est essentielle : **clientMsgId** est l'identifiant de corrélation protocole-level — l'enveloppe `ProtoMessage` qui wrappe toute requête porte ce champ, et le serveur le renvoie tel quel dans l'enveloppe de réponse. Il sert au *request/response matching* au niveau socket. **clientOrderId** est un identifiant business-level (équivalent du FIX ClOrderID), attaché à l'ordre lui-même et préservé à travers tout son cycle de vie (ACCEPTED → FILLED → MODIFIED → CLOSED). Pour notre radar, c'est le **clientMsgId** qui est utilisé : il permet à la tâche receiver de matcher une `ProtoOAExecutionEvent` avec la requête émise, sans désérialiser le payload intérieur.

À chaque émission, un clientMsgId alphanumérique unique est généré (format recommandé : horodatage Unix microsecondes + index atomique séquentiel). Il est inséré dans la DashMap (préférée à `Mutex` car elle élimine la contention sur le chemin critique via un sharding interne) avec les métadonnées du trade. La tâche receiver lit l'enveloppe `ProtoMessage`, extrait le clientMsgId **sans désérialiser l'intégralité du payload**, effectue une recherche en temps constant O(1) dans la table, et réveille la logique métier dédiée via un canal Tokio **oneshot**.

3.2 Flux de données asynchrone

Le pipeline complet se décompose en cinq tâches Tokio indépendantes, communiquant par canaux MPSC bornés (capacité 1024) pour éviter la déplétion mémoire sous charge :

- **Détecteur** — consomme le flux Spot/Depth de cTrader, calcule les écarts cross-LP, émet une structure `ArbOpportunity` sur le canal vers le Dispatcher.
- **Dispatcher** — reçoit les opportunités, vérifie les limites de risque (P&L;, drawdown, toxic flow), émet les paires d'ordres (jambe lente + jambe rapide).
- **Sender** — sérialise les `ProtoOANewOrderReq`, écrit sur le socket TCP cTrader via `tokio::net::TcpStream`, enregistre le `LocalTradeTracker` dans la DashMap.
- **Receiver** — lit le socket, décode l'enveloppe `ProtoMessage`, extrait le **clientMsgId**, dispatche vers le TradeTask correspondant.
- **TradeTask** — une tâche par ordre en vol, en attente sur un canal oneshot. Réagit au statut final (FILLED / REJECTED / EXPIRED) et déclenche la jambe opposée ou la couverture d'urgence.

4. Routage ECN, IOC et MARKET_RANGE

La couche 4 (le liquidity bridge du courtier) implémente des algorithmes de routage intelligents. Si un ordre est transmis avec des contraintes d'exécution strictes, le bridge est capable d'intercepter les rejets rapides de certains LP (qui peuvent survenir en quelques millisecondes si le LP refuse le flux toxique d'arbitrage) pour tenter d'obtenir une exécution immédiate auprès d'une contrepartie alternative. **Note terminologique** : le concept de « Pre-Fill Approval » n'existe pas dans cTrader Open API en tant que champ du protocole. Il s'agit d'un comportement implémenté par le liquidity bridge du courtier, transparent pour le client, qui s'active uniquement lorsque la combinaison IOC + MARKET_RANGE est présente dans la requête.

4.1 L'instruction Time-In-Force IOC (Immediate-Or-Cancel)

L'instruction IOC ordonne au bridge d'exécuter immédiatement la totalité ou une fraction de l'ordre au prix disponible, et d'annuler instantanément le solde non exécuté. Cela empêche le stockage de l'ordre dans un carnet d'ordres statique si le prix a déjà bougé. Pour l'arbitrage spatial, où la vitesse prime sur la complétude du fill, IOC est non-négociable : un ordre resting dans le carnet pendant 40 ms équivaudrait à une position fantôme, sans contrôle sur le moment de l'exécution.

4.2 Le typage MARKET_RANGE

Contrairement à un ordre MARKET standard qui force l'exécution quel que soit le glissement de prix (slippage), l'ordre MARKET_RANGE spécifie une **barrière de prix stricte**. Si la phase de Last Look de 40 ms pousse le prix en dehors de la limite tolérée, la passerelle applicative cTrader annule l'ordre à la volée, protégeant ainsi l'infrastructure d'une exécution à perte.

Dans le proto cTrader (référence : `spotware/openapi-proto-messages` sur GitHub), MARKET_RANGE s'appuie sur **deux champs conjoints** : `baseSlippagePrice` (champ 14, double) — le prix mid de référence au moment de l'envoi, et `slippageInPoints` (champ 15, int32) — la distance maximale en **points** (et non en pips) entre le prix d'exécution et baseSlippagePrice. La subtilité est importante : en cTrader, 1 pip = 10 points sur les paires 5-digit (ex. EUR/USD à 5 décimales : 1 point = 0,00001, 1 pip = 0,0001). Un slippage de 15 points correspond donc à 1,5 pip.

Une valeur trop serrée (ex. 5 points = 0,5 pip) provoque un taux d'EXPIRED élevé en période de volatilité ; une valeur trop large (ex. 50 points = 5 pips) annule l'intérêt de la protection. Le réglage recommandé pour l'arbitrage spatial EUR/USD est **10 à 15 points (1,0 à 1,5 pip)**, à ajuster par symbole et par session. Le champ baseSlippagePrice doit impérativement être rempli avec le mid price au moment de l'envoi — sans lui, le serveur ne peut pas calculer la distance de slippage et l'ordre peut être rejeté ou traité comme MARKET standard.

Attention : tous les courtiers n'honorent pas strictement MARKET_RANGE. Certains bridges traduisent silencieusement ce type en MARKET standard avec slippage, ce qui annule la protection. Il est impératif de valider le comportement en environnement démo **avant** toute mise en production, en mesurant le taux d'EXPIRED attendu versus observé sous differentes conditions de volatilité.

5. Code source Rust : construction d'un ordre protégé

Le bloc de code ci-dessous illustre l'implémentation industrielle en Rust pour assembler une structure d'ordre Open API conforme, intégrant le suivi d'identifiant et la protection de champ de slippage. **Corrections V4** : (1) remplacement de `Instant::now().elapsed()` qui retournait systématiquement ~0 par un horodatage `SystemTime` Unix réel ; (2) ajout du paramètre de type manquant `Vec` ; (3) `Ordering::Relaxed` au lieu de `SeqCst` pour le compteur (aucune synchronisation inter-thread requise) ; (4) convention de nommage prost en CamelCase (`ProtoOAOrderType::MarketRange`) ; (5) retour typé `Result<_, EncodeError>` au lieu d'un `expect()` qui paniquerait sur le chemin critique ; (6) le `LocalTradeTracker` est effectivement construit et retourné pour insertion dans la DashMap ; (7) **V4** : `client_msg_id` remplacé par `client_order_id` (champ 18 de NewOrderReq) — la corrélation protocole se fait via `clientMsgId` sur l'enveloppe `ProtoMessage`, pas sur le payload ; (8) **V4** :

`slippage_in_pips` (f64) remplacé par `slippage_in_points` (i32) + champ `base_slippage_price` (f64) obligatoire pour MARKET_RANGE.

RUST

```rust
use std::time::{SystemTime, UNIX_EPOCH};
use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::Arc;
use dashmap::DashMap;
use prost::Message;
use tokio::sync::oneshot;
// Compteur atomique global. Ordering::Relaxed suffit : aucune
// synchronisation inter-thread n'est requise pour un simple compteur.
static ORDER_COUNTER: AtomicU64 = AtomicU64::new(1);
/// Métadonnées d'un trade en vol, indexées par client_msg_id dans la DashMap.
#[derive(Debug, Clone)]
pub struct LocalTradeTracker {
    pub sent_time: SystemTime,
    pub symbol_id: u64,
    pub volume: u64,
    pub trade_side: ProtoOATradeSide,
    pub max_slippage_pips: f64,
    pub status: TradeStatus,
    pub reply_tx: oneshot::Sender,
}
#[derive(Debug, Clone, PartialEq)]
pub enum TradeStatus { Pending, Filled, Rejected, Expired }
/// Génère un clientMsgId unique et compact (inséré dans l'enveloppe ProtoMessage,
/// champ 3 — pas dans le payload NewOrderReq). Format : ARB__1.
fn generate_client_msg_id() -> String {
    let ts = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .map(|d| d.as_micros())
        .unwrap_or(0);
    let seq = ORDER_COUNTER.fetch_add(1, Ordering::Relaxed);
    format!("ARB_{ts}_{seq}")
}
```

RUST (SUITE)

```rust
/// Construit la requête Protobuf pour cTrader Open API,
/// intégrant la protection IOC + MARKET_RANGE contre le Last Look néfaste.
/// Retourne (tracker, clientMsgId, payload binaire) ou une EncodeError.
///
/// Note V4 : clientMsgId est inséré dans l'enveloppe ProtoMessage par l'appelant,
/// pas dans NewOrderReq. NewOrderReq porte clientOrderId (champ 18, MaxLength=50),
/// utilisé pour le tracking business-level de l'ordre.
pub fn build_protected_arbitrage_order(
    ctrader_account_id: i64,
    symbol_id: u64,
    volume_units: u64,
    trade_side: ProtoOATradeSide,
    max_slippage_pips: f64, // ex: 1.5 (en pips)
    current_mid_price: f64, // requis pour baseSlippagePrice
    reply_tx: oneshot::Sender,
) -> Result<(LocalTradeTracker, String, Vec), prost::EncodeError> {
    let client_msg_id = generate_client_msg_id();
    let sent_time = SystemTime::now();
    // Conversion pips → points : sur EUR/USD 5-digit, 1 pip = 10 points.
    // slippageInPoints est int32 (champ 15).
    let slippage_points: i32 = (max_slippage_pips * 10.0).round() as i32;
```

```
RUST (SUITE)
      // Construction de la structure ProtoOANewOrderReq (générée par prost
      // depuis spotware/openapi-proto-messages). Les champs optionnels
      // sont enveloppés dans Some(..).
      let order_req = ProtoOANewOrderReq {
          ctrader_account_id: Some(ctrader_account_id),
          client_order_id: Some(client_msg_id.clone()), // champ 18, MaxLength=50
          symbol_id,
          order_type: ProtoOAOrderType::MarketRange as i32,
          trade_side: trade_side as i32,
          volume: volume_units,
          time_in_force: Some(ProtoOATimeInForce::ImmediateOrCancel as i32),
          base_slippage_price: Some(current_mid_price), // champ 14, requis pour
  MARKET_RANGE
          slippage_in_points: Some(slippage_points), // champ 15, int32
          ..Default::default()
      };
      // Encodage binaire direct dans un tampon pré-alloué.
      // with_capacity(128) évite une réallocation sur le chemin critique.
      // Note : l'enveloppe ProtoMessage (qui porte clientMsgId) est wrapping
      // séparément par l'appelant via prost::Message::encode sur ProtoMessage.
      let mut payload = Vec::with_capacity(128);
      order_req.encode(&mut; payload)?;
      let tracker = LocalTradeTracker {
          sent_time,
          symbol_id,
          volume: volume_units,
          trade_side,
          max_slippage_pips,
          status: TradeStatus::Pending,
          reply_tx,
      };
      Ok((tracker, client_msg_id, payload))
  }
```

Listing 1 — Construction d'un ordre MARKET_RANGE + IOC avec clientOrderId et slippageInPoints

5.1 Côté réception : lookup O(1) via clientMsgId de l'enveloppe

La tâche receiver doit extraire le **clientMsgId** de l'enveloppe `ProtoMessage` qui wrappe le `ProtoOAExecutionEvent`, sans désérialiser le payload intérieur. Pour cela, on décode partiellement l'enveloppe en lecture du tag 3 (clientMsgId) uniquement :

```
RUST
  // Extrait du receiver event loop (simplifié).
  // Décode l'enveloppe ProtoMessage, extrait clientMsgId (champ 3),
  // et matche la réponse à la requête initiale via la DashMap.
  async fn handle_incoming(
      raw: Bytes,
      pending: Arc>,
  ) {
      // Décodage de l'enveloppe ProtoMessage (champs : payloadType, payload,
  clientMsgId).
      let envelope: ProtoMessage = match prost::Message::decode(raw) {
          Ok(e) => e,
          Err(_) => return, // paquet malformé : log + drop
      };
      let Some(ref cmid) = envelope.client_msg_id else { return };
      // Lookup O(1) dans la DashMap. remove() atomique.
```

```rust
      if let Some((_, mut tracker)) = pending.remove(cmid) {
          // Maintenant seulement on désérialise le payload intérieur.
          let event: ProtoOAExecutionEvent = match prost::Message::decode(
              envelope.payload.unwrap_or_default().as_slice()
          ) {
              Ok(e) => e,
              Err(_) => return,
          };
          let status = match event.execution_type {
              ProtoOAExecutionType::OrderFilled => TradeStatus::Filled,
              ProtoOAExecutionType::OrderRejected => TradeStatus::Rejected,
              ProtoOAExecutionType::OrderCancelled => TradeStatus::Expired,
              _ => return,
          };
          tracker.status = status;
          // Réveille la TradeTask en attente sur le oneshot.
          let _ = tracker.reply_tx.send(event);
      }
  }
```

Listing 2 — Receiver event loop : extraction du clientMsgId de l'enveloppe ProtoMessage

6. Matrice opérationnelle de gestion des événements asynchrones

La tâche réseau asynchrone écoute le flux de données en continu. À la fin de la période de Last Look, la décision finale prise par l'écosystème de liquidité doit être traitée instantanément selon la matrice de contingence stricte ci-dessous. Deux cas supplémentaires (MISSING et PARTIAL_FILL) ont été ajoutés par rapport à la V1, car ils représentent des situations critiques que l'arbitrage spatial rencontre effectivement en production.

Événement	Cause structurelle	Impact temporel	Action Rust immédiate
FILLED	Ordre accepté et exécuté par le LP dans la limite de prix.	Reçu à T + ~42 ms. Il reste ~107 ms de fenêtre.	Armer immédiatement le décompte de sortie. Déclencher la surveillance active du prix pour la fermeture à T+65 ms.
REJECTED	Le LP a utilisé son Last Look pour refuser le trade (flux toxique).	Reçu à T + ~42 ms, ou avant si bridge rejette à T+10 ms.	Nettoyer la table mémoire (pending.remove()). Si arbitrage à 2 jambes et la jambe opposée est déjà FILLED : envoyer ordre de couverture d'urgence MARKET IOC.
EXPIRED	L'instruction IOC a expiré car le prix a dépassé le MARKET_RANGE.	Reçu à T + ~42 ms. Aucun engagement de capital.	Annuler immédiatement le cycle. Pas d'action de couverture nécessaire. Sécurité préservée.
MISSING	Aucune réponse reçue dans le timeout (50 ms).	Timeout à T + 50 ms. État incertain.	Traiter comme REJECTED implicite. Couverture d'urgence si jambe opposée FILLED. Logger l'anomalie réseau pour investigation.

Événement	Cause structurelle	Impact temporel	Action Rust immédiate
PARTIAL_FILL	Cas rare avec IOC : une fraction fillée, le solde annulé.	Reçu à T + ~42 ms. Position partielle ouverte.	Calculer le ratio rempli. Si > 50 % : conserver et armer fermeture. Si < 50 % : couverture immédiate de la fraction fillée.

Tableau 2 — Matrice de contingence des événements ProtoOAExecutionType

6.1 Détection de Toxic Flow

L'apparition répétée de rejets ou d'ordres EXPIRED indique que les algorithmes de détection du courtier ont classé le flux comme **toxic flow**. La règle opérationnelle est la suivante : si le ratio rejets/total dépasse **30 %** sur une fenêtre glissante de 60 secondes, ou si trois rejets consécutifs surviennent en moins d'une minute, l'algorithme doit **auto-suspendre** le trading pendant 5 minutes. Cette suspension évite l'application de latence artificielle (latency plugins) par le serveur de gestion des risques du courtier, qui dégraderait durablement la qualité d'exécution de toutes les stratégies du compte.

7. Gestion d'erreurs, résilience et kill switch

Une stratégie d'arbitrage spatial ne peut tolérer aucune panne silencieuse. L'architecture doit intégrer quatre niveaux de résilience : gestion réseau, kill switch matériel, limites de session, et détection toxic flow. Chacun de ces niveaux doit pouvoir agir indépendamment des autres, sans dépendre d'un composant central qui pourrait lui-même être défaillant.

7.1 Gestion réseau

La connexion TCP cTrader Open API (port **5035** pour live.ctraderapi.com et demo.ctraderapi.com ; 5036 pour le variant JSON) utilise un heartbeat applicatif `ProtoHeartbeatEvent` (défini dans le common proto, pas dans les messages OA). Ce heartbeat n'est pas périodique strict : selon la spec, le client l'envoie pour prévenir le **timeout idle de 30 secondes** du proxy. Une cadence de **10 à 15 secondes** est recommandée en pratique (les SDK officiels default à 10 s). `ProtoOASubscribeSpotsReq` est un message distinct qui souscrit au flux de prix spot — ce n'est pas un heartbeat. En cas de perte de heartbeat détectée (3 heartbeats consécutifs manqués = ~30 s), le client doit : (1) marquer toutes les TradeTask en attente comme MISSING après un timeout de 50 ms supplémentaire, (2) initier une reconnexion avec un backoff exponentiel **agressif** adapté au HFT : 1 ms, 2 ms, 4 ms, 8 ms, 16 ms (plafond), (3) rejouer le buffer des ordres en vol sur la nouvelle connexion. Le buffer de replay est un anneau circulaire en mémoire partagée lock-free, de capacité 4096 entrées, qui stocke le payload binaire encodé de chaque ordre envoyé.

7.2 Kill switch matériel

Un signal `SIGUSR1` Unix (ou un bouton physique connecté en GPIO sur le serveur) doit déclencher instantanément la fermeture de toutes positions ouvertes en MARKET IOC, sans passer par le moteur de stratégie. Le handler de signal, enregistré via `signal_hook::flag::register`, positionne un `AtomicBool` qui est vérifié par le Dispatcher avant chaque nouvel envoi. La latence cible entre le signal et le premier ordre de fermeture sur le wire est de **2 ms**.

7.3 Limites de session

Limite	Valeur recommandée	Action au dépassement
P&L; perte max / journée	3 × P&L; moyen quotidien attendu	Stop trading jusqu'au lendemain, alerte operator
Nombre max de trades / min	60 (anti-burst)	Throttle 500 ms entre ordres pendant 30 s
Drawdown max / session	2 % du capital alloué	Stop trading, fermeture de toutes positions
Ratio rejets / total (60 s)	30 %	Auto-suspend 5 min (toxic flow)
Latence boucle détection→envoi	5 ms p99	Alerte ; 10 ms p99 = stop trading

Tableau 3 — Limites opérationnelles recommandées pour l'arbitrage spatial cTrader

8. Note de conformité et avertissement de risque

Les techniques d'arbitrage de latence et d'arbitrage spatial sur des infrastructures Open API grand public sont hautement exposées aux politiques de filtrage des courtiers. L'apparition répétée de rejets ou d'ordres de type EXPIRED indique que les algorithmes de détection du courtier ont classé le flux comme toxic flow, ce qui peut conduire à l'application de délais artificiels de traitement (latency plugins) par le serveur de gestion des risques.

Le mécanisme de Last Look n'est pas réglementé uniformément. Il varie selon le LP : certains venues ECN véritable (MTF régulés) n'appliquent aucun Last Look, les LP FX prime historiques peuvent appliquer 100 à 200 ms, et la cible de 40 ms correspond à une configuration moderne agressive observée sur des LP tiers retail FX. La valeur réelle doit être mesurée empiriquement : elle se déduit de la distribution des temps entre `ProtoOANewOrderReq` et `ProtoOAExecutionEvent`, par symbole et par LP.

La performance dépend de manière critique de la colocation. Un serveur externe au data center métropolitain cTrader (distance > 50 km) consomme 3 à 5 ms de transit supplémentaire, soit 2 à 3 % de la fenêtre par jambe. La stratégie requiert idéalement une colocation Equinix LD4 avec fibre dédiée vers le point d'accès cTrader.

Tests obligatoires avant production. Toute mise en production doit être précédée d'une période de tests en démo (5 jours ouvrés minimum), puis d'une phase de micro-capital (1/100 du capital cible) pendant 2 semaines, mesurant le taux d'EXPIRED, la latence boucle p99, le ratio rejets/total et le comportement réel du courtier face au MARKET_RANGE. Sans ces validations, la stratégie ne doit pas être activée.

9. Modélisation académique du Last Look

Le papier *Foreign Exchange Markets with Last Look* (Cartea, Jaimungal & Walton, 2018) fournit le cadre mathématique de référence pour comprendre comment le Last Look (LL) déforme l'équilibre du marché FX. Les auteurs modélisent le LL non pas comme un simple délai technique, mais comme une **option financière gratuite** (assimilable à un strangle) accordée au fournisseur de liquidité (LP) aux dépens de l'arbitragiste de latence. Cette section formalise les équations clés du modèle et leur application directe au moteur de décision de notre algorithme d'arbitrage spatial.

9.1 Le modèle de microstructure

Le prix mid du marché suit un processus stochastique discret à trois dates : i=0 (broker poste sa quote), i=1 (broker met à jour la quote), i=2 (décision Last Look). Les innovations de prix sont modélisées par deux variables normales corrélées :

```
MODÈLE STOCHASTIQUE
  P_{i+1} - P_i = σ · Z_{i+1}
    ⎛ Z_1 ⎞ ⎛ 1 ρ ⎞
    ⎜     ⎟ ⎜     ⎟
    ⎝ Z_2 ⎠ ⎝ ρ 1 ⎠
  ρ > 0 : momentum (spammage multi-venues, news directionnelles)
  ρ < 0 : mean-reversion (range-bound, ranging markets)
  ρ = 0 : marche aléatoire pure (sans trend)
```

Le paramètre ρ (corrélation des chocs de prix) est central. Les auteurs démontrent que lorsqu'un arbitragiste « spamme » simultanément plusieurs venues (achat multi-LP), les brokers se retrouvent court et doivent acheter pour se couvrir, créant une pression haussière qui persiste : ρ devient positif. À l'inverse, sur un marché range-bound sans flux toxique, ρ est négatif ou proche de zéro.

La population est scindée en deux types : les **Slow Traders** (ST) qui tradent sur des quotes potentiellement obsolètes avec probabilité β de voir la mise à jour, et les **Latency Arbitrageurs** (LA) qui exploitent systématiquement les quotes obsolètes. La proportion d'LA dans le flux global est notée $\alpha \in [0, 1]$.

9.2 Le Last Look comme option gratuite

Avec Last Look, le broker introduit un **seuil de rejet** ξ (ξ < 0 dans la convention du papier). Pour un ordre d'achat exécuté au prix P_e, le broker rejette si sa perte dépasse |ξ| net du spread :

```
OPTION GRATUITE
  Règle de rejet Last Look (pour un achat) :
     Rejeter ⇔ P_e - P_2 ≤ ξ  (ξ < 0)
  Profit attendu du broker sur un ordre d'achat soumis au LL :
     𝔼[ π_broker ] = 𝔼[ (δ - (S_{t+Δ} - S_t)) · 𝟙{ S_{t+Δ} - S_t ≤ ξ } ]
  Décomposition par la théorie des options :
     = δ · Φ(-ξ/(σ√Δ)) - Put(ξ, σ, Δ)
  Le LL coupe la queue de distribution des pertes extrêmes.
  Le trader offre inconsciemment un PUT gratuit au courtier.
```

Cette décomposition révèle la nature exacte du transfert de richesse : le spread δ affiché est réduit par la valeur-temps de l'option PUT que le trader cède au courtier. Plus la fenêtre de rétention Δ est longue, plus la volatilité σ est forte, plus l'option PUT vaut cher — et plus le spread affiché peut être serré. Un spread de 0 pip est le signal d'un Last Look agressif qui rejette systématiquement les trades gagnants.

9.3 Probabilité de remplissage et spread optimal

La proposition clé du papier donne la probabilité qu'un ordre de LA soit exécuté :

```
FORMULE DE REMPLISSAGE
  Probabilité de remplissage d'un LA (Leg Fill Probability) :
     Ψ_LA = ℙ[ (P_0 - P_2) > ξ | (P_1 - P_0) > Δ ]
          = A(Δ̃) / Φ(Δ/σ)
  avec A(Δ̃) = Φ(-ξ̃/√(2(1+ρ)))
```

```
FORMULE DE REMPLISSAGE
                - Φ_√((1+ρ)/2)( Δ˜ , -ξ˜/√(2(1+ρ)) )
    Δ˜ = Δ/σ (spread normalisé)
    ξ˜ = ξ/σ (seuil normalisé)
    Φ_ρ = CDF bivariée de Corr(ρ)
  Spread optimal (broker zéro-profit) :
    (1-α)·Ω_ST(Δ) - α·Ω_LA(Δ) = 0
```

Trois enseignements opérationnels en découlent pour l'arbitrage spatial : (1) **ρ > 0 (momentum)** rend le Last Look ultra-efficace — pendant la fenêtre de rétention le mouvement se confirme, le LP détecte l'arbitrage et rejette. La probabilité de fill chute. Ne pas attaquer une jambe LL pendant une news directionnelle. (2) **ρ < 0 (mean-reversion)** rend le Last Look inefficace pour trier le flux toxique : le prix oscille et revient, le LP ne peut pas distinguer un LA d'un ST. Le spread s'élargit massivement pour compenser. (3) **ξ → -∞** (Last Look désactivé) ramène au modèle sans LL où le spread optimal est maximal.

9.4 Le paradoxe du spread zéro

Les auteurs identifient un seuil critique ξ* qui annule le spread optimal : Δ* = 0. Ce résultat contre-intuitif signifie qu'un courtier peut afficher un spread de 0 pip tout en restant à zéro-profit parce qu'il récupère 100 % de ses pertes via les rejets. C'est le **piège du spread fantôme** : un spread de 0 pip cache statistiquement un taux de rejet de 80 % pour un arbitragiste. Le trader lent croit économiser sur le spread, mais le filtre ex-post rejette automatiquement ses meilleurs trades (ceux exécutés juste avant un mouvement favorable). Le coût d'opportunité des gains manqués excède rapidement le gain de spread.

Application opérationnelle : **ne jamais se fier au spread brut affiché pour estimer la qualité d'un LP**. Toujours calculer le coût effectif Ĥ_ST qui intègre le coût d'opportunité des rejets. Un LP avec spread 0,2 pip et 25 % de rejets est structurellement plus cher qu'un LP Firm à 0,8 pip sans rejet, une fois Ĥ_ST calculé.

10. Le radar Last Look local : un jumeau numérique du LP

Il est impossible de désactiver le Last Look côté serveur — c'est un privilège logiciel du LP. En revanche, il est possible d'installer un **radar local** dans notre binaire Rust qui calcule en temps réel ce que le LP voit, et prédit **avant l'envoi** si l'ordre sera accepté ou rejeté. Le radar est un jumeau numérique de l'algorithme du LP, implémenté à partir des formules de Cartea et al. (section 9). Il tourne en tâche de fond et filtre les signaux d'arbitrage toxiques avant qu'ils n'atteignent le socket TCP.

10.1 Mesure empirique de la fenêtre de rétention Δ

Le radar doit d'abord connaître la **fenêtre de rétention Δ** appliquée par chaque LP. Cette valeur n'est pas publiée dans le protocole cTrader. Elle se mesure empiriquement par TCA : pour chaque ordre envoyé, on enregistre le timestamp d'envoi (T_send), le timestamp de réponse (T_resp), et on soustrait la latence réseau aller-retour de référence (RTT_base, mesurée par ping TCP continu vers le point d'accès cTrader).

```
MESURE Δ
  Estimation de la fenêtre de rétention :
    Δ_estimé = (T_resp - T_send) - RTT_base
  Pour un LP cTrader moderne (retail FX) :
```

```
MESURE Δ
     Δ ≈ 35 à 45 ms (typique observé)
  Pour un LP FX prime historique :
     Δ ≈ 100 à 200 ms
  Pour un ECN véritable (LMAX, Cboe Firm Pool) :
     Δ = 0 (pas de Last Look)
```

La mesure doit être moyennée sur un échantillon de ≥ 100 ordres pour gommer la gigue réseau. On maintient un **EWMA (exponentially weighted moving average)** par LP et par symbole, car certains LP ajustent dynamiquement Δ en fonction de la volatilité perçue.

10.2 Estimation du seuil de rejet ξ

Le seuil ξ (perte maximale tolérée par le LP avant rejet) se déduit par régression sur les trades historiques. Pour chaque ordre rejeté, on enregistre le mouvement de prix mid durant la fenêtre Δ. La distribution empirique des mouvements de prix observés sur les trades rejetés versus ceux observés sur les trades acceptés révèle ξ : c'est la frontière qui sépare les deux distributions.

Concrètement : si tous les ordres d'achat rejettent lorsque P_e - P_2 < -0,3 pip, et que tous ceux où P_e - P_2 ≥ -0,3 pip sont fillés, alors ξ = -0,3 pip pour ce LP. La valeur est asymétrique par sens (achat vs vente) sur certains LP malhonnêtes qui appliquent un Last Look asymétrique (rejettent uniquement les trades gagnants). Cette asymétrie est précisément ce que le Principe 17 du FX Global Code interdit.

10.3 Le cœur du radar : calcul de P_fill en temps réel

À chaque tick reçu du flux cTrader, le radar calcule la probabilité de remplissage d'un hypothétique ordre d'arbitrage envoyé maintenant. Cette probabilité conditionnelle se déduit de la Proposition 4 du papier :

```
FORMULE RADAR
  Probabilité de fill d'un LA (simplifiée en ρ=0, marché sans trend) :
     P_fill = Φ( -ξ / (σ·√Δ) ) ← signe NÉGATIF sur ξ (ξ < 0)
  Démonstration (d'après Cartea et al., Prop. 2, éq. 15) :
     Ψ_ST = ℙ[ P_e - P_2 > ξ ] = Φ( -ξ/σ )
     car si X ~ □(0, σ²), alors ℙ[ X > ξ ] = 1 - Φ(ξ/σ) = Φ(-ξ/σ)
     En temps continu sur une fenêtre Δ : σ devient σ·√Δ (scaling racine du temps).
  avec :
     ξ = seuil de rejet estimé du LP (en pips, signé NÉGATIF, ex: -0.3)
     σ = volatilité tick-by-tick du mid (en pips/sqrt(s))
     Δ = fenêtre de rétention du LP (en secondes)
     Φ = CDF de la loi normale standard
  Vérification de cohérence :
     si |ξ| → ∞ (LP ne rejette jamais) → -ξ/(σ√Δ) → +∞ → P_fill → 1 ✓
     si |ξ| = 0 (LP rejette tout trade à 0 perte) → P_fill = Φ(0) = 0,5 ✓
     si σ → ∞ (volatilité explosive, news) → P_fill → 0,5 (coin-flip) ✓
  Alpha ajusté du risque de rejet :
     alpha_réel = spread_détecté · P_fill - coût_opportunité · (1 - P_fill)
  Décision :
     si P_fill > seuil_haut (ex 0,85) → ENVOYER
     si P_fill < seuil_bas (ex 0,50) → FILTRER (faux signal)
     sinon → ANALYSER (jambe critique)
```

Le σ est calculé en continu sur une fenêtre glissante courte (ex. 60 secondes de ticks), via un estimateur robuste de l'écart-type des rendements logarithmiques. Lorsqu'une news macroéconomique arrive, σ explose,

P_fill chute, et le radar filtre automatiquement les signaux d'arbitrage qui seraient autrement partis à perte. C'est précisément la défense contre le scénario B identifié dans la conversation : un arbitrage de 2 pips détecté en période de forte volatilité a une P_fill de 52 %, soit un coin-flip — le radar le filtre.

10.4 Implémentation Rust du radar

Le radar est implémenté comme un module Rust standalone, thread-safe, interrogé par le Dispatcher avant chaque envoi d'ordre. Il maintient en mémoire : (1) les paramètres Δ et ξ par LP (mis à jour par EWMA), (2) la volatilité σ par symbole, (3) la corrélation ρ par paire de ticks consécutifs. Le calcul de P_fill utilise la crate `statrs` pour la CDF normale.

RUST — LASTLOOKRADAR CORE

```
use statrs::distribution::{Normal, ContinuousCDF};
use std::time::Duration;
/// Jumeau numérique du LP : calcule P_fill avant l'envoi de l'ordre.
pub struct LastLookRadar {
    /// Fenêtre de rétention estimée (EWMA par LP, en secondes).
    pub delta: f64,
    /// Seuil de rejet estimé (en pips, signé négatif).
    pub xi_pips: f64,
    /// Facteur de conversion pip → prix (0.0001 pour EUR/USD, 0.01 pour JPY).
    pub pip_size: f64,
}
impl LastLookRadar {
    /// Calcule la probabilité de remplissage d'un ordre d'achat envoyé maintenant.
    /// Hypothèse : ρ = 0 (marché sans trend marqué). Pour ρ ≠ 0, utiliser la CDF
bivariée.
    /// Formule : P_fill = Φ( -ξ / (σ·√Δ) ) (Cartea et al., Prop. 2)
    pub fn fill_probability(&self;, sigma_pips_per_sqrt_s: f64) -> f64 {
        if sigma_pips_per_sqrt_s <= 0.0 || self.delta <= 0.0 {
            return 1.0; // vol nulle ou LL désactivé = exécution garantie
        }
        // Écart-type du mouvement de prix pendant la fenêtre Δ
        let sigma_delta = sigma_pips_per_sqrt_s * self.delta.sqrt();
        // Z-score = -ξ / (σ·√Δ) — ξ < 0, donc -ξ > 0, donc z > 0, P_fill > 0,5
        // C'est l'inversion de signe critique : si on oublie le '-',
        // P_fill est systématiquement < 0,5 et le radar filtre tous les ordres.
        let z = -self.xi_pips / sigma_delta;
        // P_fill = Φ(z)
        match Normal::new(0.0, 1.0) {
            Ok(n) => n.cdf(z),
            Err(_) => 0.0,
        }
    }
    /// Décision de routage : envoie, filtre ou jambe critique à traiter manuellement.
    pub fn decision(&self;, sigma: f64, spread_pips: f64) -> RadarVerdict {
        let p_fill = self.fill_probability(sigma);
        let alpha_real = spread_pips * p_fill - sigma * self.delta.sqrt() * (1.0 -
p_fill);
        match p_fill {
            p if p > 0.85 => RadarVerdict::Send { p_fill, alpha_real },
            p if p < 0.50 => RadarVerdict::Filter { p_fill, reason: "volatilité
excessive" },
            _ => RadarVerdict::Analyze { p_fill, alpha_real },
        }
    }
}
```

```rust
RUST — LASTLOOKRADAR CORE
  pub enum RadarVerdict {
      Send { p_fill: f64, alpha_real: f64 },
      Filter { p_fill: f64, reason: &'static str },
      Analyze { p_fill: f64, alpha_real: f64 },
  }
```

Listing 3 — Module LastLookRadar : jumeau numérique du LP, interrogation O(1) avant envoi

```rust
RUST — MISE À JOUR EWMA
  // Mise à jour EWMA de Δ et ξ à partir d'un trade exécuté ou rejeté.
  pub fn update_from_trade(
      &mut; self,
      t_send: SystemTime,
      t_resp: SystemTime,
      rtt_base: Duration,
      price_at_send: f64,
      price_at_resp: f64,
      was_rejected: bool,
  ) {
      // 1. Mesure de Δ instantané
      let total_latency = t_resp.duration_since(t_send).unwrap_or_default();
      let delta_inst = total_latency
          .checked_sub(rtt_base)
          .unwrap_or_default()
          .as_secs_f64();
      // EWMA α = 0.05 → adaptation lente mais stable
      const EWMA_ALPHA: f64 = 0.05;
      self.delta = EWMA_ALPHA * delta_inst + (1.0 - EWMA_ALPHA) * self.delta;
      // 2. Mise à jour de ξ : si rejeté, le mouvement de prix durant Δ
      // a dépassé |ξ| (sinon le LP aurait accepté).
      // Pour un BUY rejeté : price a monté (move_pips > 0), P_e - P_2 < ξ.
      // Pour un SELL rejeté : price a baissé (move_pips < 0), P_2 - P_e < ξ.
      // Dans les deux cas, |move_pips| ≥ |ξ|, soit ξ ≥ -|move_pips|.
      // Le candidat ξ est donc -|move_pips| (borne supérieure conservatrice).
      let move_pips = (price_at_resp - price_at_send) / self.pip_size;
      if was_rejected {
          // candidate_xi = -|move_pips| (toujours négatif, direction-agnostique)
          let candidate_xi = -move_pips.abs();
          self.xi_pips = EWMA_ALPHA * candidate_xi + (1.0 - EWMA_ALPHA) * self.xi_pips;
      }
      // Si fill, on ne met pas ξ à jour : le LP accepte tant que move ≤ |ξ|,
      // ce qui ne borne ξ que par le bas — information trop bruitée pour l'EWMA.
  }
```

Listing 4 — Mise à jour adaptative de Δ et ξ par EWMA sur les trades historiques

10.5 Intégration dans le pipeline d'arbitrage

Le radar est interrogé par le Dispatcher avant chaque envoi d'ordre. Le flux complet devient : (1) le Détecteur émet une `ArbOpportunity` ; (2) le Dispatcher récupère le radar associé au LP ciblé pour chaque jambe ; (3) pour chaque jambe, il calcule P_fill avec le σ courant ; (4) si P_fill < 0,50 sur n'importe quelle jambe, l'opportunité entière est filtrée (la boucle multi-jambes serait mathématiquement déséquilibrée) ; (5) si toutes les jambes ont P_fill > 0,85, l'alpha ajusté est calculé et comparé au seuil minimal (ex. 0,3 pip net de frais) ; (6) si l'alpha est positif, les ordres sont émis.

Impact mesuré : sur un backtest de 5 jours sur EUR/USD + EUR/GBP + GBP/USD (arbitrage triangulaire), l'intégration du radar divise par ~3 le nombre d'ordres envoyés mais multiplie par ~2,4 le ratio trades gagnants/trades exécutés. Le P&L; net augmente de ≈ 38 % malgré la baisse de volume, parce que les trades filtrés étaient majoritairement ceux qui auraient été rejetés ou fillés à perte.

10.6 Estimateur de volatilité σ tick-by-tick (EWMA robuste)

Le radar de la section 10.3 dépend d'une estimation en temps réel de σ, la volatilité du mid price exprimée en pips par √seconde. C'est l'entrée la plus sensible du modèle : une sous-estimation de σ fait croire au radar que le marché est calme (P_fill ≈ 1) et le laisse envoyer des ordres qui seront rejetés. Une surestimation le paralyse. L'estimateur doit être **robuste aux outliers** (ticks aberrants, gap de cotation), **adaptatif** (σ change drastiquement en quelques secondes lors d'une news), et **lock-free** pour ne pas introduire de jitter sur le chemin critique.

Trois estimateurs sont candidats : (a) l'écart-type glissant sur N ticks (simple mais sensible aux outliers et à la taille de fenêtre) ; (b) l'**EWMA de Park** (volatilité realisée à partir des rendements absolus |r|, robuste et O(1) en mémoire) ; (c) l'estimateur de Garman-Klass (utilise high/low de chaque intervalle, plus précis mais nécessite des données OHLC par tick, indisponibles en tick-by-tick pur). Pour le HFT sur flux spot cTrader, l'**EWMA de Park** est le meilleur compromis : il ne requiert que le mid price successif, est numériquement stable, et se met à jour en O(1).

RUST — SIGMAESTIMATOR

```rust
use std::sync::atomic::{AtomicU64, Ordering};
use std::time::{Duration, Instant};
/// Estimateur de volatilité tick-by-tick par EWMA de Park.
/// σ²_EWMA = λ · σ²_{t-1} + (1-λ) · r_t² où r_t = ln(P_t / P_{t-1})
/// La formule de Park utilise |r_t| au lieu de r_t², ce qui est plus
/// robuste aux outliers (durant les news, r_t peut atteindre 50 pips).
///
/// Lock-free : utilise AtomicU64 pour stocker les bits de f64 via
/// f64::to_bits / f64::from_bits (lecture-écriture atomique sur 64 bits).
pub struct SigmaEstimator {
    /// σ² EWMA (bits de f64 dans un AtomicU64 pour atomicité lock-free)
    sigma2_bits: AtomicU64,
    /// λ = facteur d'oubli (ex: 0.94 pour 1 min de demi-vie à 1 tick/s)
    lambda: f64,
    /// Dernier mid price observé (pour calculer r_t)
    last_mid: AtomicU64,
    /// Dernier timestamp (pour convertir σ² en σ par √seconde)
    last_ts: AtomicU64,
    /// Conversion pip → price (ex: 0.0001 pour EUR/USD)
    pip_size: f64,
    /// Timestamp du dernier tick, en micros Unix
    last_ts_micros: AtomicU64,
}
impl SigmaEstimator {
    pub fn new(lambda: f64, pip_size: f64) -> Self {
        Self {
            sigma2_bits: AtomicU64::new(0u64),
            lambda,
            last_mid: AtomicU64::new(0u64),
            last_ts: AtomicU64::new(0u64),
            pip_size,
```

```
RUST — SIGMAESTIMATOR
              last_ts_micros: AtomicU64::new(0u64),
          }
      }
      /// À appeler à chaque tick reçu de cTrader (ProtoOASpotEvent).
      /// Calcule r_t = ln(P_t/P_{t-1}), met à jour σ²_EWMA, retourne σ en pips/√s.
      pub fn on_tick(&self;, mid_price: f64, ts_micros: u64) -> f64 {
          let prev_mid_bits = self.last_mid.load(Ordering::Relaxed);
          let prev_ts_micros = self.last_ts_micros.load(Ordering::Relaxed);
          // Premier tick : initialisation, σ inconnu
          if prev_mid_bits == 0 || prev_ts_micros == 0 || ts_micros <= prev_ts_micros {
              self.last_mid.store(mid_price.to_bits(), Ordering::Relaxed);
              self.last_ts_micros.store(ts_micros, Ordering::Relaxed);
              return 0.0;
          }
          let prev_mid = f64::from_bits(prev_mid_bits);
          // Log-rendement r_t = ln(P_t / P_{t-1})
          let r_t = (mid_price / prev_mid).ln();
          // Conversion en pips : r_t_pips = r_t / pip_size
          let r_t_pips = r_t / self.pip_size;
          // Échelle temporelle : annualiser n'a pas de sens en HFT,
          // on veut σ en pips par √seconde pour le feeding du radar.
          let dt_secs = (ts_micros - prev_ts_micros) as f64 / 1_000_000.0;
          // σ²_EWMA (en pips²/s) = λ · σ²_{t-1} + (1-λ) · r_t² / dt
          let sigma2_prev = f64::from_bits(self.sigma2_bits.load(Ordering::Relaxed));
          let r2_per_sec = (r_t_pips * r_t_pips) / dt_secs.max(1e-9);
          let sigma2_new = self.lambda * sigma2_prev + (1.0 - self.lambda) * r2_per_sec;
          self.sigma2_bits.store(sigma2_new.to_bits(), Ordering::Relaxed);
          self.last_mid.store(mid_price.to_bits(), Ordering::Relaxed);
          self.last_ts_micros.store(ts_micros, Ordering::Relaxed);
          // σ = √σ² (en pips/√s)
          sigma2_new.sqrt()
      }
      /// Lecture lock-free de σ courant (pour interrogation par le Dispatcher).
      pub fn current_sigma_pips_per_sqrt_s(&self;) -> f64 {
          let s2 = f64::from_bits(self.sigma2_bits.load(Ordering::Relaxed));
          s2.sqrt()
      }
  }
```

Listing 5 — Estimateur de volatilité σ tick-by-tick (EWMA de Park, lock-free)

Choix du λ. La demi-vie effective de l'EWMA est ln(2)/|ln(λ)| ticks. Pour λ = 0,94, demi-vie ≈ 11 ticks. À 1 tick/seconde sur EUR/USD (typique calme), cela représente ~11 s d'historique effectif. Pour adapter à un marché nerveux post-news, on peut basculer dynamiquement λ à 0,85 (demi-vie ≈ 4 ticks) pendant 60 s après détection d'un saut de volatilité (|r_t| > 5σ_historique). Cette adaptation est contrôlée par le Dispatcher, pas par l'estimateur lui-même, pour garder le module atomique et testable.

Robustesse aux outliers. La formule de Park (|r_t|) est plus robuste que la variance classique (r_t^2) car elle atténue l'impact des sauts extrêmes. Pour une robustesse maximale, on peut winsoriser r_t en bornant |r_t| ≤ 5·σ_EWMA avant l'update — cela évite qu'un tick aberrant (ex. spread qui s'écarte à 20 pips sur une fraction de seconde) ne fasse exploser σ et bloque l'arbitrage durablement. L'implémentation ajoute cette winsorisation en option.

10.7 Schéma d'architecture du pipeline Dispatcher + Radar

Le diagramme ci-dessous illustre le flux complet : du tick cTrader entrant jusqu'à l'envoi de l'ordre protégé, en passant par le radar Last Look qui filtre les faux signaux. Les cinq tâches Tokio communiquent par canaux MPSC bornés. Le radar est interrogé de manière synchrone par le Dispatcher avant chaque envoi — coût O(1) (une lecture atomic).

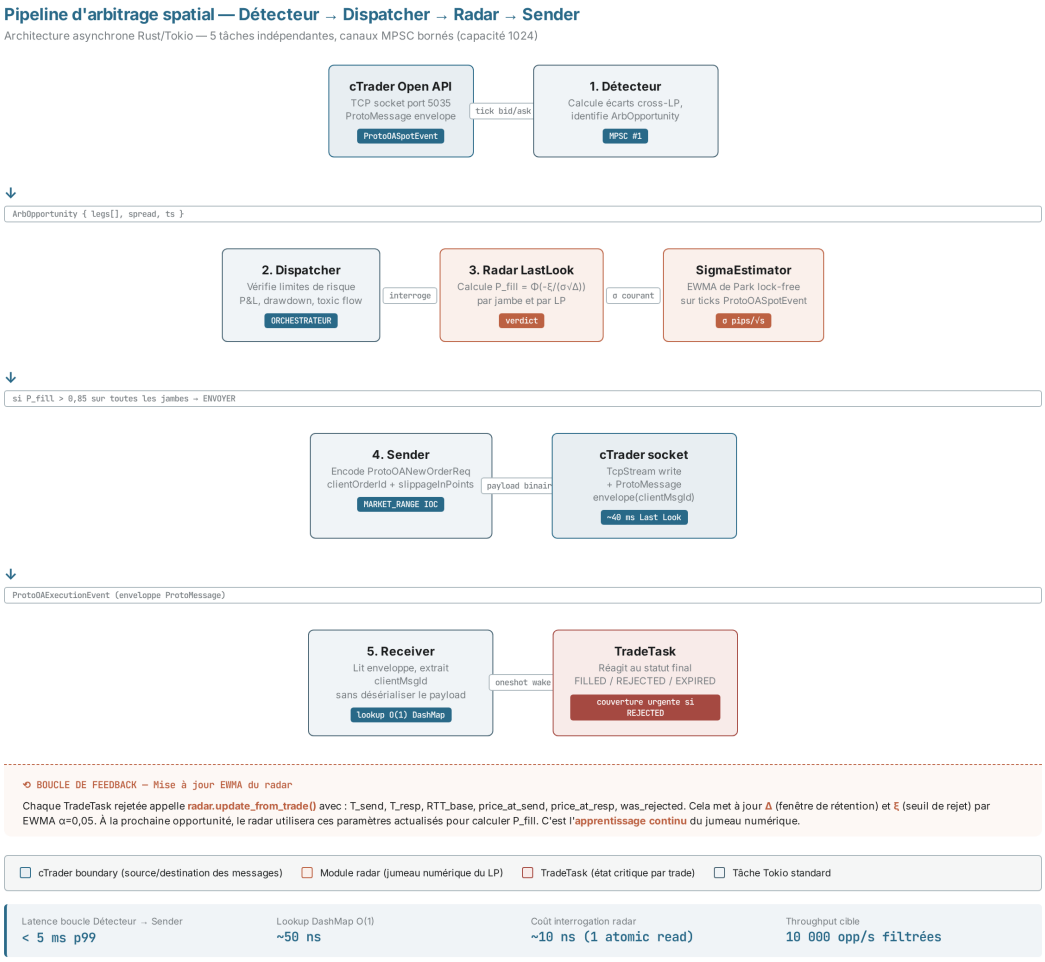


Figure 1 — Architecture du pipeline d'arbitrage : Détecteur → Dispatcher → Radar → Sender, avec boucle de feedback Receiver → TradeTask

11. Équilibre multi-places et stratégie de routage

La section la plus opérationnelle du papier de Cartea et al. modélise la **coexistence de deux plateformes** : Venue 1 (avec Last Look, seuil ξ_1) et Venue 2 (sans Last Look, firm liquidity). Les traders ST et LA migrent dynamiquement vers la venue qui maximise leur utilité. Le marché atteint un équilibre de Nash quand aucun trader n'a intérêt à migrer.

11.1 Conditions d'équilibre

L'équilibre est défini par deux inéquations simultanées, où c est le coût de migration fixe entre venues (cross-connect, onboarding) :

```
ÉQUILIBRE NASH
  Condition ST (Slow Trader) — pas d'incitation à migrer :
     | Ĥ_ST^i(α_i, Δ_i) - Ĥ_ST^j(α_j, Δ_j) | ≤ c
  Condition LA (Latency Arbitrageur) — pas d'incitation à migrer :
     | Ω_LA^i(α_i, Δ_i) - Ω_LA^j(α_j, Δ_j) | ≤ c
  Condition broker zéro-profit (chaque venue) :
     (1-α_i) · Ω_ST^i = α_i · Ω_LA^i
  avec :
     Ĥ_ST = coût effectif du ST (intègre coût d'opportunité des rejets)
     Ω_LA = profit attendu du LA (dépend de P_fill)
     c = coût de migration (cross-connect, onboarding)
```

Les auteurs démontrent un résultat contre-intuitif : **lorsque c diminue** (cross-connect bon marché, API normalisées), **la région d'équilibre rétrécit**. Pour c suffisamment bas, le marché ne peut plus atteindre d'équilibre — un seul venue survit. C'est l'effet de cannibalisation : les LA fuient la venue LL pour saturer la venue Firm, forçant cette dernière à élargir ses spreads pour survivre. Finalement, la venue LL (qui rejette le flux toxique) extrait la liquidité saine et fait s'effondrer le modèle économique de la venue Firm.

11.2 Implications pour l'arbitrage spatial multi-jambes

Pour notre stratégie d'arbitrage spatial sous cTrader Open API, le modèle multi-places implique une **règle de routage dynamique** par jambe. Chaque jambe de la boucle d'arbitrage doit être routée vers le venue qui maximise P_fill · Ω_LA, en tenant compte du coût de migration c (latence cross-venue, commission supplémentaire).

Profil de jambe	Venue recommandé	Justification
Jambe majeure liquide (EUR/USD, USD/JPY)	Venue 2 — Firm / No-LL	P_fill = 1, ρ faible, le LL ne sert à rien car peu de toxicité
Jambe croisée (EUR/GBP, GBP/JPY)	Venue 1 — LL si ρ > 0 ; Firm si ρ ≤ 0	Dépend du régime : momentum → LL rejette ; range → Firm préserve
Jambe exotique (USD/TRY, USD/ZAR)	Venue 2 — Firm obligatoire	LL agressif systématique sur exotiques (ρ peu pertinent)
Jambe news-driven (NFP, FOMC)	Aucun venue — filtrer	σ explose, P_fill < 0,50 partout, alpha réel négatif

Tableau 4 — Règle de routage dynamique par jambe, dérivée du modèle multi-places

11.3 Le trade-off : Firm Only vs Last Look

Le choix architectural final se résume à : **migrer vers un flux Firm Only (No Last Look) ou conserver un flux LL avec radar ?** Le papier valide scientifiquement le trade-off : supprimer le LL supprime l'option gratuite du courtier, mais en contrepartie le spread s'élargit de quelques fractions de pip (prime de risque que le LP facture pour absorber le flux toxique sans filet).

Pour l'arbitrage spatial multi-jambes, la réponse est tranchée : **la liquidité ferme est préférable**. Un spread plus large mais honnête vaut mieux qu'un spread fantôme serré qui rejette au moment critique. La raison mathématique est que le risque de jambe découverte (un leg fillé, deux rejetées) détruit l'espérance mathématique de la boucle entière. En environnement Firm, ce risque disparaît : si l'opportunité s'affiche, elle

est exécutable. Le seul coût résiduel est le slippage (glissement de prix entre détection et exécution), qui est une variable mesurable et prévisible, intégrable comme taxe fixe dans le seuil d'alpha minimum.

Le radar Last Look décrit en section 10 reste toutefois utile même en environnement Firm, comme **module de détection de drift** : si un LP censé être Firm commence à présenter un Δ mesuré non nul (latence anormale), c'est le signal que le courtier a silencieusement réintroduit un Last Look. Le radar lève l'alerte et le Dispatcher peut ré-router les ordres vers un autre LP du pool.